Assignment No. 6

Name : Mohit Sojal **PRN :** 124B1F027

Batch: A2 **Div :** A **Course :** Design and Analysis of Algorithms

Github Link : https://github.com/mohitsojal1/DAA--Assignment/tree/main

Disaster Relief Resource Allocation Using 0/1 Knapsack

1. Problem Statement

A massive earthquake has struck a remote region, and a relief organization must transport essential supplies to the affected population. The organization has a relief truck with a limited carrying capacity of (W) kilograms and (N) different types of relief items. Each item has a specific weight and a utility value representing its importance in meeting the urgent needs of affected people.

The objective is to select a combination of items that maximizes the total utility without exceeding the truck's weight capacity.

The problem is modelled using the **0/1 Knapsack Problem**, in which each item can either be included in the truck or excluded from it. Items cannot be divided into smaller portions.

The solution uses Dynamic Programming to determine the optimal selection of relief items. The model can also be extended to consider life-saving supplies, perishable goods, multiple trucks, and changing relief requirements.

2. Tasks

1. Model disaster relief resource allocation as a 0/1 Knapsack Problem.
2. Implement an algorithm to maximize the total utility of selected items without exceeding the truck's capacity.
3. Identify the items selected for transportation.
4. Analyze the time and space complexity of Brute Force, Dynamic Programming, and Greedy approaches.
5. Discuss how priority-based selection, multiple trucks, and real-time updates can improve disaster relief operations.

3. Course Objectives

1. To understand the computational complexity of various algorithms.
2. To select appropriate algorithm design strategies for solving real-world problems.

4. Course Outcomes

1. Analyze the asymptotic performance of algorithms.
2. Generate optimal solutions by applying the Dynamic Programming strategy.

5. Theory

The objective is to maximize the total utility of selected items without exceeding the truck's maximum weight capacity.

Objective Function:

Maximize: $Z = \Sigma(value[i] \times x[i])$, for $i = 1$ to N

Weight Constraint:

$\Sigma(weight[i] \times x[i]) \leq W$, for $i = 1$ to N

Item Selection Constraint:

$x[i] \in \{0, 1\}$

Where:

- N = Total number of items.
- W = Maximum truck capacity.
- weight[i] = Weight of item i.
- value[i] = Utility value of item i.
- x[i] = 1 if item i is selected; otherwise, x[i] = 0.
- Z = Total utility of the selected items.

5.2 Dynamic Programming Approach

DP State:

dp[i][w] = Maximum utility obtainable using the first i items with a capacity limit of w.

Base Cases:

dp[0][w] = 0, for all w from 0 to W.

dp[i][0] = 0, for all i from 0 to N.

Recurrence Relation:

If weight[i-1] ≤ w:

dp[i][w] = max(value[i-1] + dp[i-1][w - weight[i-1]], dp[i-1][w])

Otherwise:

dp[i][w] = dp[i-1][w]

Final Maximum Utility:

Maximum Utility = dp[N][W]

5.3 Utility-to-Weight Ratio for the Greedy Approach

Ratio[i] = value[i] / weight[i]

Items are sorted in descending order of this ratio for the Fractional Knapsack Problem. This approach does not guarantee an optimal solution for the 0/1 Knapsack Problem.

6. Algorithmic Approaches

6.1 Brute Force Approach

The Brute Force approach examines every possible subset of the available items.

Working:

1. Generate all possible subsets of the (N) items.
2. Calculate the total weight and utility of each subset.
3. Discard subsets whose total weight exceeds the truck's capacity.
4. Identify the feasible subset with the highest utility.
5. Return the maximum utility and the corresponding items.

Since every item can either be selected or rejected, there are (2^N) possible subsets.

Time Complexity: (O(N2^N)) for a straightforward implementation that evaluates the weight and utility of every subset.

Space Complexity: (O(N)) auxiliary space when subsets are generated recursively without storing every subset.

Advantages:

- Simple to understand and implement.
- Guarantees an optimal solution when all subsets are evaluated correctly.

Disadvantages:

- Computationally expensive for large numbers of items.
- The number of possible subsets grows exponentially.

Therefore, Brute Force is most suitable for small problem instances.

6.2 Dynamic Programming Approach

Dynamic Programming is an optimization technique that solves a problem by dividing it into smaller subproblems and storing their results to avoid repeated calculations.

It is effective for the 0/1 Knapsack Problem because the problem has optimal substructure and overlapping subproblems.

DP State

Let:

[
dp[i][w]
]

represent the maximum utility obtainable using the first (i) items when the available capacity is (w).

Base Cases

If no items are available or the capacity is zero, the maximum utility is zero.

[
dp[0][w]=0
]

[
dp[i][0]=0
]

Recurrence Relation

If the weight of the current item is less than or equal to the available capacity, there are two possibilities:

1. Include the current item.
2. Exclude the current item.

Therefore,

[
dp[i][w]=\max
\begin{cases}
value[i-1]+dp[i-1][w-weight[i-1]],\
dp[i-1][w]
\end{cases}
]

If the item's weight exceeds the available capacity, it cannot be included:

[dp[i][w]=dp[i-1][w]]

The final maximum utility is:

[\boxed{dp[N][W]}]

Working:

1. Initialize a DP table of size ((N+1)\times(W+1)) with zeros.
2. Process the items one by one.
3. For every item, evaluate all capacities from zero to (W).
4. If the item fits, compare the utility obtained by including it with the utility obtained by excluding it.
5. Store the larger value in the DP table.
6. Obtain the maximum utility from (dp[N][W]).
7. Trace backward through the DP table to identify the selected items.

Time Complexity: (O(NW)).

Space Complexity: (O(NW)) for the two-dimensional DP table.

Advantages:

- Guarantees an optimal solution for the standard 0/1 Knapsack Problem.
- Avoids repeated evaluation of the same subproblems.
- Determines both the maximum utility and the selected items.
- Is generally more efficient than Brute Force for moderate item counts and capacities.

Limitation:

The complexity (O(NW)) is pseudopolynomial rather than polynomial in the complete input size because it depends on the numerical value of the capacity (W). Very large capacities may require substantial time and memory.

6.3 Greedy Approach

The Greedy approach makes decisions based on a locally attractive choice at each step.

For the Fractional Knapsack Problem, items are sorted by their utility-to-weight ratio:

[Ratio[i]=\frac{value[i]}{weight[i]}]

Items with the highest ratios are selected first. If the remaining capacity cannot accommodate an entire item, a fraction of that item can be included.

Time Complexity: (O(N\log N)), primarily due to sorting.

Advantages:

- Simple and efficient for Fractional Knapsack.
- Works well when items can be divided.

Limitation for 0/1 Knapsack:

In the 0/1 Knapsack Problem, items cannot be divided. Selecting items solely by their utility-to-weight ratio does not guarantee the maximum total utility.

Therefore, Dynamic Programming is the preferred approach for obtaining an optimal solution to the disaster relief problem as defined.

6.4 Comparison of Approaches

Feature	Brute Force	Dynamic Programming	Greedy
Main technique	Enumerates subsets	Solves and stores subproblems	Makes locally attractive choices
Time complexity	(O(N2^N))	(O(NW))	(O(N\log N)) with sorting
Optimal for 0/1 Knapsack	Yes	Yes	Not guaranteed
Memory requirement	(O(N)) auxiliary space for recursive enumeration	(O(NW))	Typically (O(N)) additional space
Scalability	Poor for large (N)	Depends on (N) and (W)	High
Best application	Small instances	Standard 0/1 Knapsack	Fractional Knapsack

7. Working of the Algorithm

The Dynamic Programming algorithm follows these steps.

Step 1: Input

Read the number of items (N), the weight and utility value of each item, and the truck's maximum capacity (W).

Step 2: Initialize the DP Table

Create a two-dimensional table `dp[N+1][W+1]` and initialize every entry to zero.

Step 3: Fill the DP Table

For every item (i), consider each capacity (w) from zero to (W).

- If the current item is too heavy, copy the result from the previous row.
- Otherwise, compare the result of including the item with the result of excluding it.
- Store the larger utility value.

Step 4: Find the Maximum Utility

The value stored in `dp[N][W]` is the maximum achievable utility.

Step 5: Identify Selected Items

Start from `dp[N][W]` and examine the table backward.

If `dp[i][w]` differs from `dp[i-1][w]`, item (i) was selected in the reconstructed solution. Add its weight to the selected-item total and reduce the remaining capacity by its weight.

Continue until all items have been examined or the remaining capacity becomes zero.

Step 6: Display the Result

Display the maximum utility, the selected items, their total weight, and the remaining truck capacity.

8. Algorithm: 0/1 Knapsack Using Dynamic Programming

Input:

- Number of items (N).
- Weight array `weight[]`.
- Utility array `value[]`.
- Truck capacity (W).

Output:

- Maximum achievable utility.
- List of selected items.

1. Create a DP table `dp[N+1][W+1]` and initialize it to zero.
2. For each item (i) from 1 to (N):
    - For each capacity (w) from 1 to (W):
        - If `weight[i-1] <= w`, calculate the maximum of including and excluding the item.
        - Otherwise, copy the value from the previous row.
3. Set `maxUtility = dp[N][W]`.
4. Initialize `w = W` and an empty list of selected items.
5. For each item (i) from (N) down to 1:
    - If `dp[i][w] != dp[i-1][w]`, add item (i) to the selected-item list and subtract its weight from (w).
6. Return the maximum utility and selected-item list.

9. Pseudocode

```
KNAPSACK(weight, value, N, W)

  Create dp[N+1][W+1]
  Initialize all entries of dp to 0

  for i = 1 to N:
    for w = 1 to W:

      if weight[i-1] <= w:

        include = value[i-1]
```

```
                + dp[i-1][w-weight[i-1]]

            exclude = dp[i-1][w]

            dp[i][w] = max(include, exclude)

        else:
            dp[i][w] = dp[i-1][w]

maxUtility = dp[N][W]

selectedItems = empty list
w = W

for i = N down to 1:

    if dp[i][w] != dp[i-1][w]:

        Add item i to selectedItems
        w = w - weight[i-1]

return maxUtility, selectedItems
```

10. Priority-Based Resource Allocation

In disaster relief operations, some supplies may be more urgent than others. Medicines, drinking water, and emergency food may require higher priority than less urgent supplies.

A basic way to represent importance is to assign a utility value to each item based on its expected benefit.

For example:

- Essential medicines may receive high utility values because they are needed to treat critical injuries.
- Drinking water may receive a high utility value when clean water is scarce.
- Food packets may receive higher utility values when the affected population lacks access to meals.
- Nonessential supplies may receive lower utility values when transportation capacity is limited.

The Dynamic Programming algorithm then selects the combination with the highest total utility within the weight constraint.

Important consideration: Increasing an item's utility can influence its selection, but it does not guarantee that the item will be selected. If a particular item is mandatory, the model must explicitly enforce that requirement or solve a suitably constrained optimization problem. Perishability can also be represented through urgency-adjusted utility values, provided the scoring system is defined consistently.

11. Time and Space Complexity

Let (N) be the number of items and (W) be the truck's capacity.

Measure	Complexity	Description
Time Complexity	(O(NW))	Each item is evaluated for every possible capacity.
Space Complexity	(O(NW))	A two-dimensional DP table stores the results of the subproblems.
Space-Optimized DP	(O(W))	A one-dimensional DP array can calculate the maximum utility without storing the entire table.
Brute Force Time	(O(N2^N))	All subsets are evaluated in a straightforward implementation.
Greedy Time	(O(N\log N))	Sorting by utility-to-weight ratio dominates the Fractional Knapsack approach.

The Dynamic Programming solution is suitable when the item count and capacity are manageable. Its pseudopolynomial complexity means that performance depends on the numerical capacity, not just the number of items.

12. Conclusion

The Disaster Relief Resource Allocation problem demonstrates how the 0/1 Knapsack Problem can be applied to distribute limited transportation capacity among essential supplies.

Dynamic Programming determines the combination of items that maximizes total utility without exceeding the truck's weight limit. By storing intermediate results in a DP table, the algorithm avoids the repeated calculations associated with Brute Force and guarantees an optimal solution for the standard 0/1 Knapsack model.

The approach can be extended to consider urgency-based utility values, perishable supplies, multiple trucks, and changing resource requirements. Although its (O(NW)) complexity is pseudopolynomial, it provides an effective solution for many practical resource-allocation problems.

Thus, Dynamic Programming supports systematic, efficient, and utility-based planning for disaster relief operations.
